MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 1 – Lab

Program: MSc in AI & Entrepreneurship

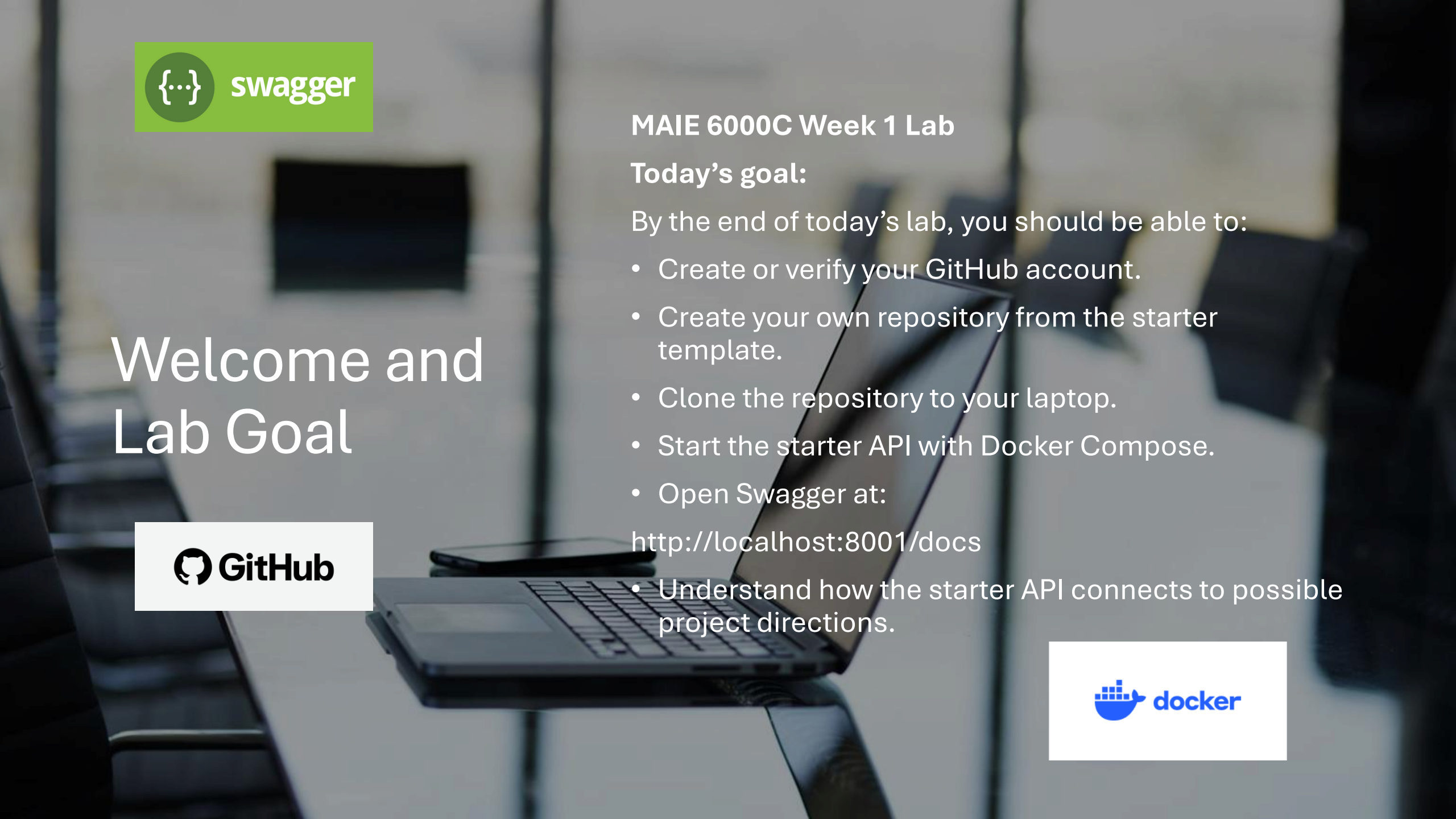
Term: Fall 2026/27

Instructor: Lefteris NTAFLOS, PhD

The Hong Kong University of Science and Technology

Session 1 — Setup, GitHub, Docker, and First Run

Session goal: By the end of Session 1, we should have our own copy of the starter repository cloned locally and should be able to start the system with Docker Compose.



Welcome and Lab Goal

MAIE 6000C Week 1 Lab

Today's goal:

By the end of today's lab, you should be able to:

- Create or verify your GitHub account.
- Create your own repository from the starter template.
- Clone the repository to your laptop.
- Start the starter API with Docker Compose.
- Open Swagger at:

http://localhost:8001/docs

- Understand how the starter API connects to possible project directions.

What We Are Proving Today

Today we want to prove that:

- You can access GitHub.
- You can create your own copy of the starter repository.
- You can clone the repository locally.
- Docker and Docker Compose work on your machine (install Docker Desktop).
- The API starts successfully.
- The database, worker, and AI service start successfully.
- You can submit a case and see it processed.

Important: Do not wait until Week 3 to discover that setup is broken.

Required Accounts and Local Tools

Accounts you need

- Canvas access
- HKUST email access
- GitHub account

Software to have locally

- Git
- Docker Desktop or equivalent Docker + Compose setup
- Code editor, such as VS Code or Sublime Text
- Optional: Python 3.11 for editor support and local convenience

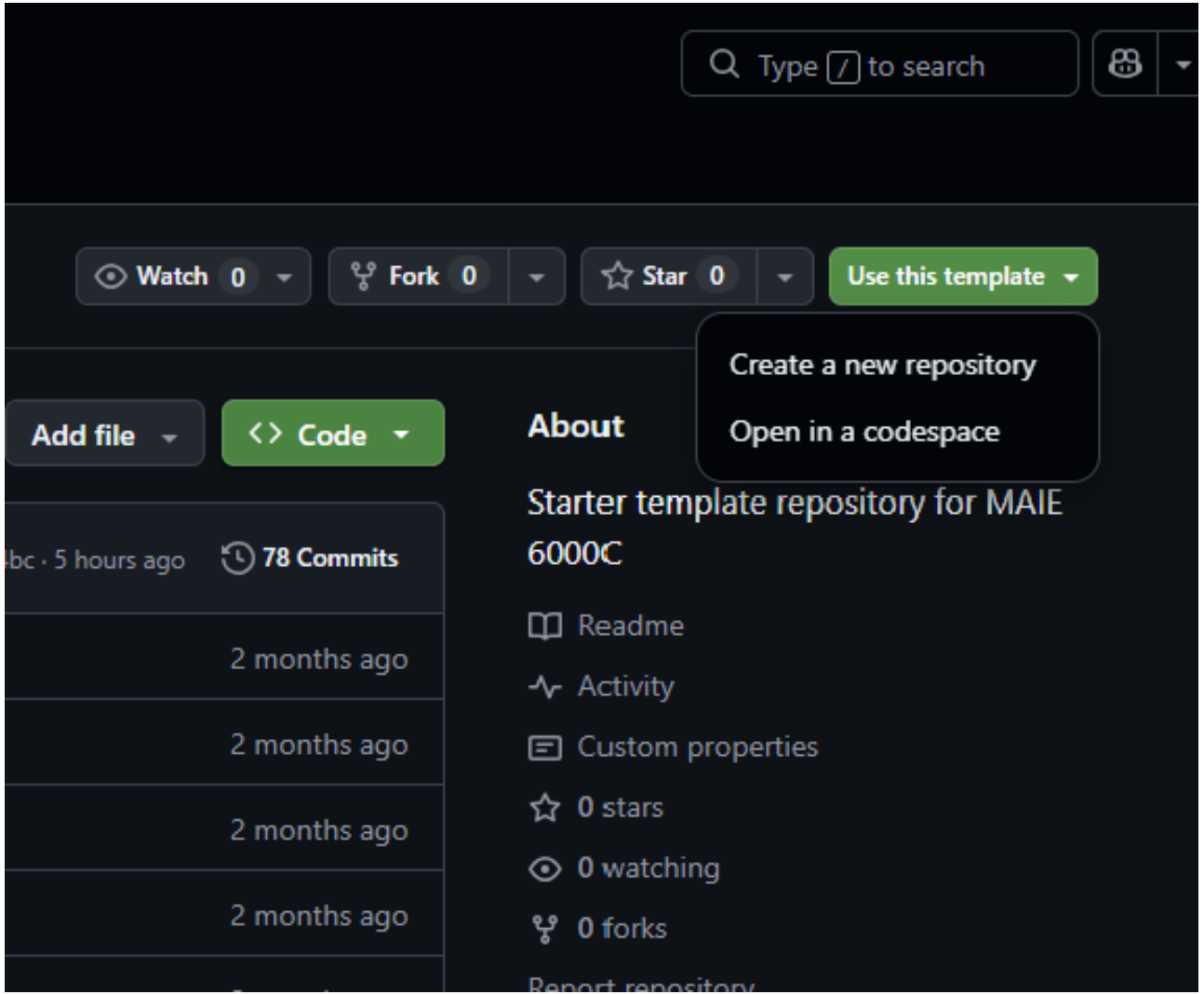
Create Your Own Repository from the Starter Template

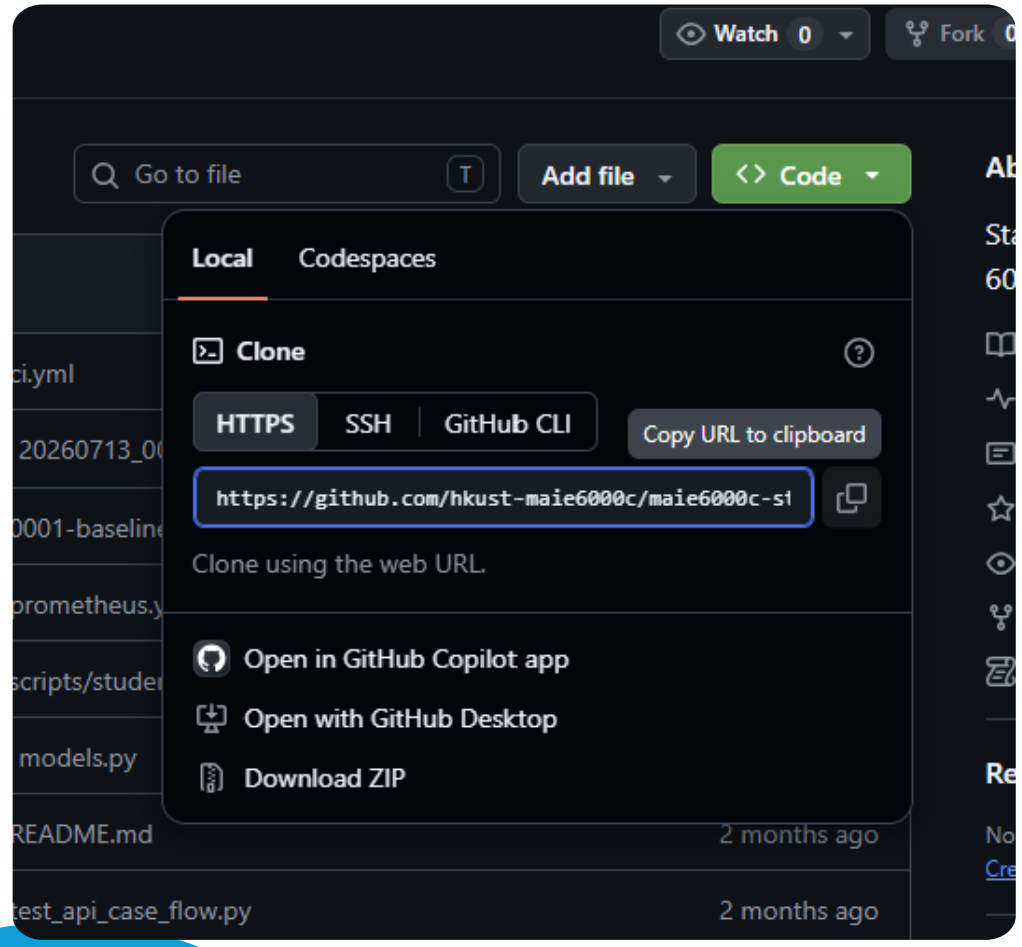
Go to the starter template:

- https://github.com/hkust-maie6000c/maie6000c-starter-template

Then

1. Click **Use this template**.
2. Choose **Create a new repository**.
3. Select your GitHub account as the owner unless instructed otherwise.
4. Choose a repository name.
5. Create the repository.





Clone Your Repository to Your Machine

On your new GitHub repository page:

- Click **Code**.
- Copy the HTTPS clone URL.
- Open PowerShell, Terminal, or Git Bash.
- Navigate to where you keep course files.
- Clone your repository.

Example:

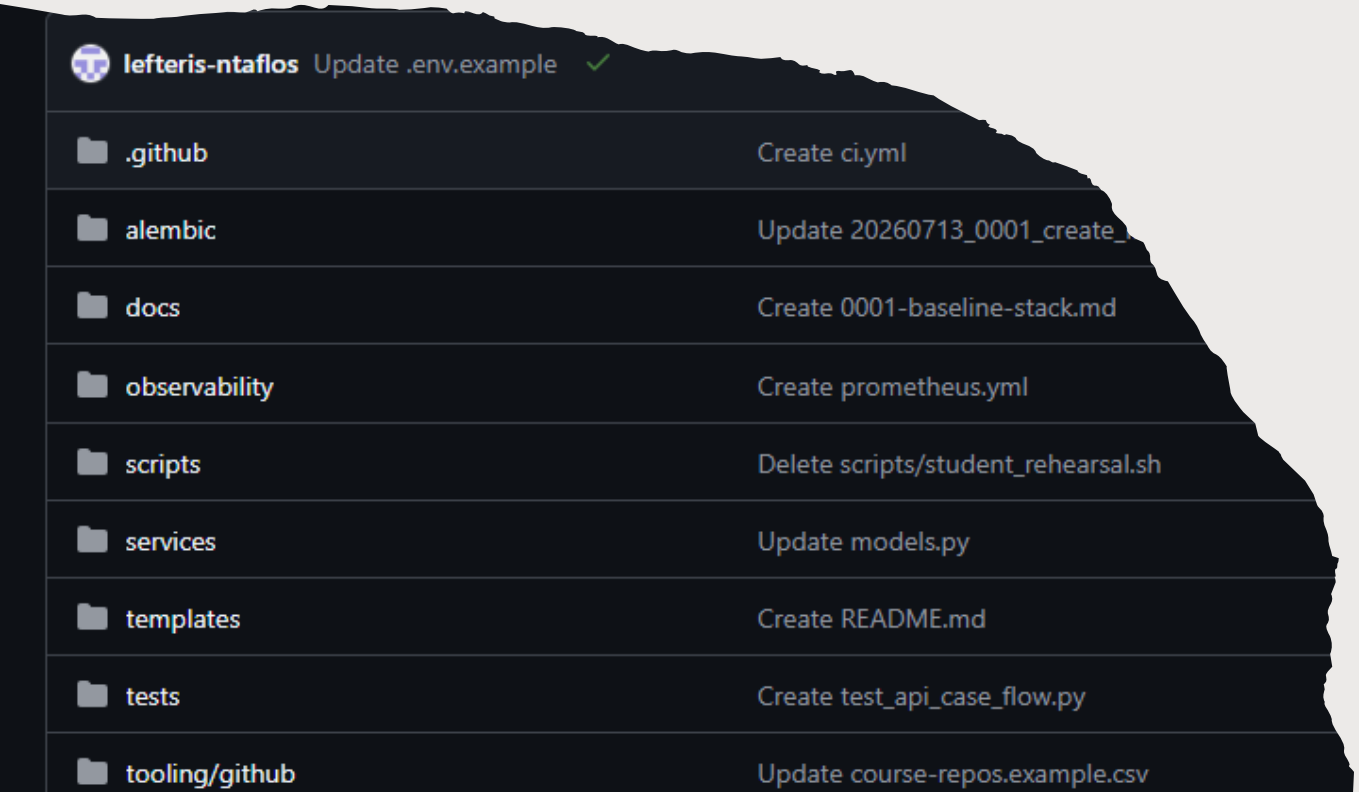
Mac Terminal

```
git clone https://github.com/YOUR-USERNAME/YOUR-REPO-NAME.git
cd YOUR-REPO-NAME
```

Windows PowerShell

```
cd C:\Users\YourName\Documents
git clone https://github.com/YOUR-USERNAME/YOUR-REPO-NAME.git
cd YOUR-REPO-NAME
```

What Is the Starter Template?



The starter template is a small multi-service backend application.

It can:

- Accept a new case from a user.
- Save that case.
- Create a background job.
- Send the case to a mock AI triage service.
- Receive an AI result.
- Store the result.
- Let you read the case and job status through an API.

Simple version:

- **Submit a text case → AI triage service labels it → result is stored and readable through the API.**

The Four Main Services

The starter system runs multiple services:

Service	Role
api	Receives HTTP requests
db	Stores cases, jobs, and AI results
worker	Processes background jobs
ai	Mock AI triage service

You may see logs like:

```
api-1
worker-1
ai-1
db-1
```

Each prefix shows which container produced the log line.

Docker and Docker Compose in Plain English

Docker

- Runs applications in isolated environments called containers.
- **Container**

A running copy of an image.

Concept	Simple meaning
Docker image	Recipe or template
Docker container	Running copy of that recipe

Docker Compose
Runs multiple containers together.
For this app, Compose starts:

- text
- api
- db
- worker
- ai

Docker Desktop and Git Installation - Quick Local Verification

Install Docker Desktop and Git

Run these commands from your Mac terminal or Windows PowerShell:

```
git --version
docker --version
docker compose version
```

Expected idea:

- Git should print a version number.
- Docker should print a version number.
- Docker Compose should print a version number.
- If Docker commands fail, start Docker Desktop and try again.

Prepare the Environment File

From inside the repository folder, create your .env file.

For macOS:

```
cp .env.example .env
```

For Windows PowerShell:

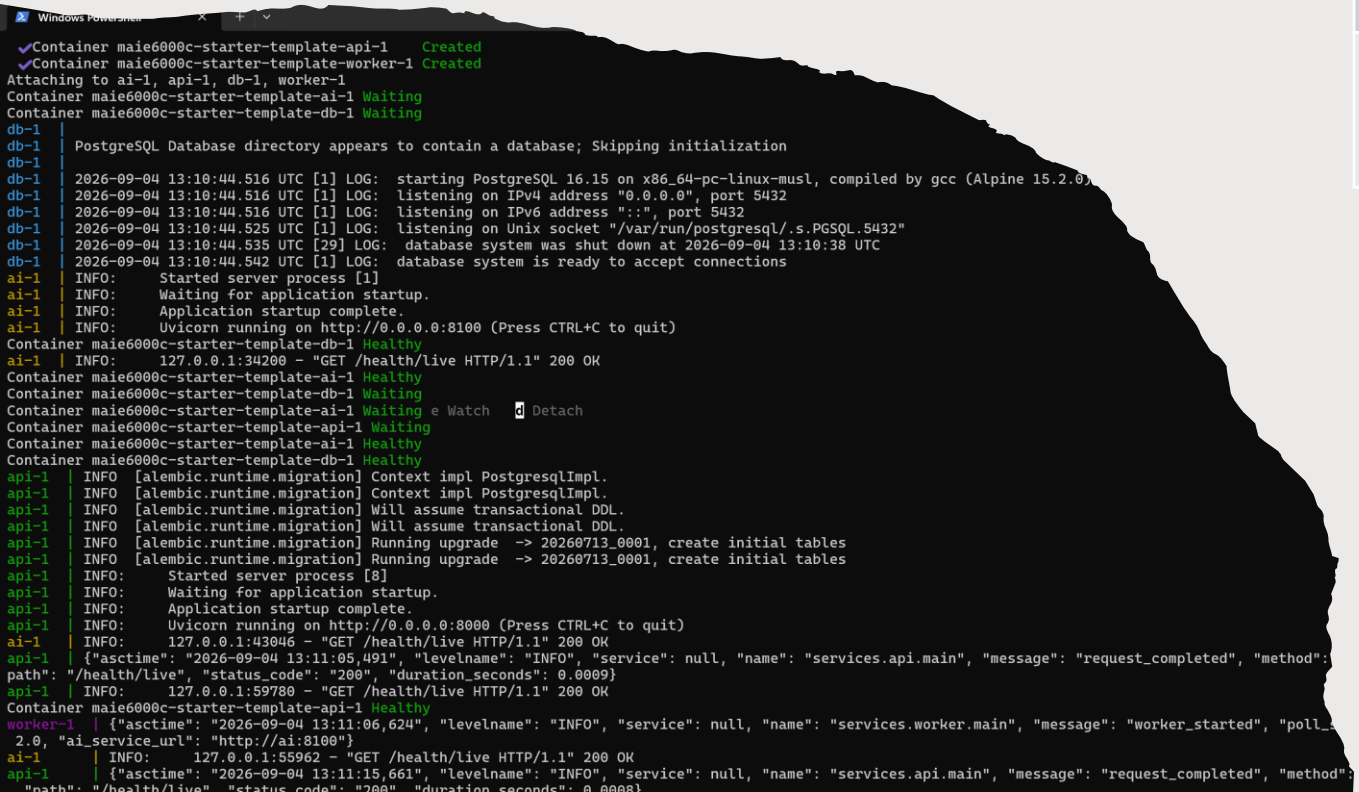
```
Copy-Item .env.example .env
```

Then check the Compose configuration:

```
docker compose config
```

This checks whether Docker Compose can read the project configuration.

Start the App



Run:

```
docker compose up --build
```

Part	Meaning
`docker`	Use Docker
`compose`	Use Docker Compose
`up`	Start the application
`--build`	Rebuild images before starting

Leave this terminal open.

You should see logs from:

```
api-1
worker-1
ai-1
db-1
```

Session 1 Checkpoint: Open Swagger

When the logs show the API is running, open:

- http://localhost:8000/docs

or

- http://localhost:8001/docs
  check your .env file

You should see the Swagger API documentation page.

Swagger lets you:

- See available endpoints.
- Test requests in the browser.
- View server responses.
- Confirm the starter API is running.

Checkpoint: Keep Docker Compose running.

Session 2 — API Walkthrough and Project Direction Activity

Session goal: By the end of Session 2, we should understand the starter workflow and produce an initial project-direction note.

Session 2 Restart and Checkpoint

Before continuing, check:

1. Docker Compose is still running.

2. Swagger is open:
`http://localhost:8000/docs`

3. You can see endpoints such as:
- `GET /`
- `GET /health/live`
- `GET /health/ready`
- `POST /cases`
- `GET /cases`
- `GET /cases/{case_id}`
- `GET /jobs/{job_id}`

If the app is stopped, run again:
`docker compose up --build`

Swagger, Endpoints, and HTTP Methods

Swagger
- Interactive API documentation.

Endpoint
- A URL path that performs an action.

Examples:
- GET /cases
- POST /cases
- GET /jobs/{job_id}

In this starter pack:
- GET /cases = read cases
- POST /cases = create a case

Method	Meaning
GET	Read information
POST	Create something new
PUT	Replace something
PATCH	Modify part of something
DELETE	Delete something

Test the Root and Health Endpoints

In Swagger, test:

```
GET /
```

Expected response:

```
json
{
  "service": "api",
  "status": "ok"
}
```

Then test:

```
GET /health/live
GET /health/ready
```

Endpoint	Meaning
`/health/live`	Is the API process alive?
`/health/ready`	Is the API ready to handle real requests?

Create a Case with POST /cases

In Swagger, open:

`POST /cases`

Click: Try it out

Use this JSON body:

```
{
  "title": "Payment issue",
  "description": "Customer reports that their credit card was charged twice after submitting an order. They need help with refund status and confirmation."
}
```

Click: Execute

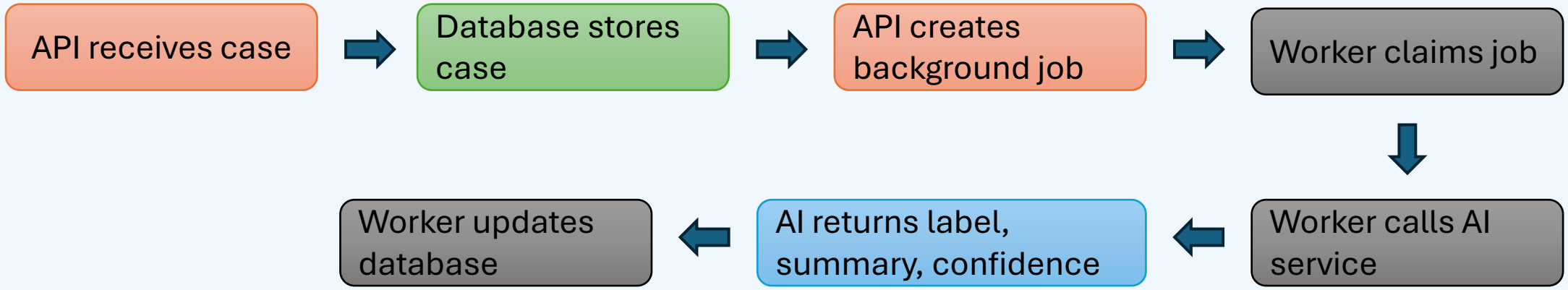
Expected status: `201 Created`

The response should include:

- A case
- A job

What Happens After You Create a Case?

After POST /cases:



Look for logs such as:

```
case_created
job_claimed
POST /triage
triage_completed
job_completed
```

Read the Case and Job Status

Now test

```
GET /cases
```

Expected idea:

```
[
  {
    "title": "Payment issue",
    "status": "triaged",
    "ai_label": "general",
    "ai_confidence": 0.45
  }
]
```

Then test:

```
GET /jobs/1
```

Expected idea:

```
{
  "id": 1,
  "status": "completed",
  "error": null
}
```

Exact IDs and values may vary.

Same Test Using PowerShell or Mac Terminal

PowerShell

Check root

```
Invoke-RestMethod
http://localhost:8001/
```

Check readiness

```
Invoke-RestMethod
http://localhost:8001/health/ready
```

Create a case

```
$body = @{

 title = "Payment issue"

  description = "Customer reports
that their credit card was charged
twice after submitting an order.
They need help with refund status
and confirmation."

} | ConvertTo-Json

Invoke-RestMethod `

  -Uri http://localhost:8001/cases
`

  -Method POST `

  -ContentType "application/json" `

  -Body $body
```

List cases

```
Invoke-RestMethod
http://localhost:8001/cases
```

Mac Terminal

Check root

```
curl http://localhost:8001/
```

Check readiness

```
curl
http://localhost:8001/health/ready
```

Create a case

```
curl -X POST
http://localhost:8001/cases \

  -H "Content-Type:
application/json" \

  -d '{

    "title": "Payment issue",

    "description": "Customer
reports that their credit card was
charged twice after submitting an
order. They need help with refund
status and confirmation."

  }’
```

List cases

```
curl http://localhost:8001/cases
```

Stopping, Restarting, and Resetting

Command	Meaning
`ps`	Show running services
`logs`	Show logs
`logs -f`	Follow live logs
`down`	Stop and remove containers/network
`down -v`	Also delete volumes/data

To stop the running app:
`Ctrl + C`

Then optionally run:
```
docker compose down
```

To remove stored local database data too:
```
docker compose down -v
```

Use -v carefully.

Useful troubleshooting commands:
```
docker compose ps
docker compose logs
docker compose logs -f
```

Bridge: From Starter API to Course Projects

The starter API demonstrates a reusable project pattern.

Starter concept	Project meaning
Case	A submitted item, request, report, or record
POST /cases	User submits something
Database	System remembers the submitted data
Job	Work that should happen later
Worker	Background processor
AI triage	AI-enabled classification, summary, ranking, or extraction
GET /cases	User or admin reads results
Human review	Someone checks or overrides the AI output

Form Pairs or Provisional Teams

Activity: Form pairs or provisional teams

Instructions:

1. Work in pairs or provisional teams.
2. Open the **Project Brief Pack**
3. Choose 2–3 briefs to review.
4. Do not choose the broadest idea.
5. Look for something that can become a small working demo by Week 7.

Team discussion starter:

- What kind of user has a real problem here?
- What would they submit into the system?
- What would the system store?
- What would the AI step do?

Review 2–3 Project Briefs

For each brief, ask:

1. Who exactly is the user?
2. What is the first thing that enters the system?
3. What table or record must exist?
4. What work should not happen inside the request cycle?
5. What can be shown by Week 7?
6. What is too broad?

Choose one candidate brief for today's note.

Instructor / TA role: Circulate and ask narrowing questions.

Project-Direction Note Template

Each group should produce a short note answering:

Candidate brief:

Primary user:

Core workflow:

System input:

Persisted data:

Async/worker step:

AI-enabled function:

Human review/fallback:

Smallest Week 7 demo:

Biggest scope risk:

Example mapping:

System input: user submits a complaint

Persisted data: complaint record and status

Async/worker step: classify complaint

AI-enabled function: label complaint category and summarize

Human review/fallback: staff member confirms or edits label

Smallest Week 7 demo: submit complaint and see AI label

Expected Output and Wrap-Up

By the end of Week 1 lab, each student or provisional team should have:

- One or two candidate project directions.
- A draft workflow shape.
- An initial sense of what the starter repository provides.
- A clear next technical step.

Minimum successful lab outcome:

```
I can run the starter API.

I understand the case → job → worker → AI → result flow.

I can map that flow to one possible project idea.
```